

PDF exclusivo para André Pires de Oliveira Capela - rm376905
andr.capela@gmail.com

SUMÁRIO

O QUE VEM POR AÍ? 3
HANDS ON 4
SAIBA MAIS 5
O QUE VOCÊ VIU NESTA AULA? 18
REFERÊNCIAS 19

FIAP

PDF exclusivo para André Pires de Oliveira Capela - rm376905
andr.capela@gmail.com

O QUE VEM POR AÍ?

Vamos desvendar o potencial transformador dos Grandes Modelos de Linguagem (LLMs) na construção de aplicações práticas, um marco na era da inteligência artificial conversacional.

Você entenderá questões técnicas e operacionais dos LLMs, aprendendo sobre as três principais formas de integrá-los em aplicações: execução local (On-Premises) para máxima segurança e privacidade, alocação em servidores de nuvem para escalabilidade e flexibilidade e, por fim, a assinatura de serviços que oferecem acesso a modelos pré-treinados. A última opção emerge como a mais viável para a maioria das empresas, dada a alta demanda por recursos computacionais e os custos, muitas vezes proibitivos, associados ao treinamento e inferência de LLMs.

Finalmente, concretizamos nosso aprendizado por meio da criação de um assistente virtual acadêmico para alunos da FIAP, capaz de oferecer suporte em uma vasta gama de tópicos acadêmicos e consultas ao guia de normas da instituição. Este exemplo prático ilustra a aplicação de engenharia de prompts e recuperação de documentos em ação, democratizando o acesso a tecnologias de ponta e abrindo novos horizontes para o desenvolvimento de soluções inteligentes em educação e outras áreas.

PDF exclusivo para André Pires de Oliveira Capela - rm376905
andr.capela@gmail.com

HANDS ON

Nesta aula, exploraremos a implementação prática de Grandes Modelos de Linguagem (LLMs) em projetos, especificamente aplicada a chatbots e assistentes virtuais. Abordaremos a transição desde os conceitos teóricos até as aplicações reais, destacando a importância de compreender a base teórica e os desenvolvimentos históricos dos LLMs para aproveitar plenamente seu potencial. Discutiremos as três estratégias principais para a integração de LLMs em aplicações: execução local para máxima privacidade, alocação em servidores de nuvem para flexibilidade e escalabilidade e o uso de APIs de modelos pré-treinados para acessibilidade e efetividade de custo.

A aula enfatizará o desafio do alto custo computacional associado aos LLMs, tanto para treinamento quanto para inferência, e como isso restringe a adoção de modelos avançados pelas empresas de pequeno e médio porte, especialmente no Brasil. Destacamos a solução prática de recorrer a APIs de provedores como OpenAI e Hugging Face, que disponibilizam o acesso a essas tecnologias avançadas, permitindo que desenvolvedores de todos os níveis criem aplicações sofisticadas sem a necessidade de infraestrutura dedicada. Os custos do uso dessas APIs também serão discutidos.

Ao longo da aula, mergulharemos nos aspectos técnicos e práticos dos LLMs, preparando vocês para enfrentar os desafios e aproveitar as oportunidades que essas tecnologias oferecem no campo da inteligência artificial conversacional.

Tenha uma ótima aula!

FIAP

PDF exclusivo para André Pires de Oliveira Capela - rm376905
andr.capela@gmail.com

SAIBA MAIS

Parabéns por chegar ao final da nossa jornada, desvendando o poder das LLMs. Nesse momento final da disciplina, vamos discutir um pouco mais sobre uma questão crucial a respeito das LLMs: o seu custo computacional, que se traduz invariavelmente no custo financeiro para sua utilização. Ao final desta seção, você terá uma boa estimativa do custo operacional e poderá responder a uma pergunta que deve ter feito várias vezes ao longo das aulas: eu consigo rodar uma LLM no meu PC?

O custo de processamento de redes neurais é um tema fundamental tanto na pesquisa quanto na aplicação de inteligência artificial. A evolução dessas tecnologias trouxe avanços significativos, mas também diversos desafios relacionados ao poder computacional necessário para treinar e manter uma aplicação. Como vocês devem lembrar, existiam inicialmente as Redes Neurais Multicamadas (MLPs, do inglês "Multi-Layer Perceptrons") que representaram um marco na capacidade de aprender representações de dados de maneira hierárquica e possuíam capacidade de resolver diversos problemas das mais diversas áreas. Embora as MLPs tenham sido cruciais para os avanços iniciais em campos como o reconhecimento de padrões e classificação, rapidamente ficaram evidentes limitações quanto ao seu custo computacional e eficiência ao lidar com dados de alta dimensionalidade, como imagens e sequências de texto. Basicamente, a quantidade de neurônios necessários para lidar com esse tipo de dado tornava os treinamentos muito longos, caros e, de uma forma geral, não geravam resultados tão bons quanto ao lidar com dados de baixa dimensionalidade, como previsão de preços de produtos ou probabilidades de vendas (embora sejam utilizadas nas mais diversas previsões).

Para lidar com essas questões, foram desenvolvidas as Redes Neurais Convolucionais (CNNs, do inglês "Convolutional Neural Networks"), gerando uma revolução na forma como os dados visuais são processados. CNNs introduziram mecanismos eficientes para a detecção de características em imagens, reduzindo significativamente a quantidade de parâmetros necessários quando comparadas a MLPs. Isso não apenas melhorou a precisão na tarefa de reconhecimento de imagens, mas também otimizou o custo de processamento ao explorar a natureza espacial dos dados visuais.

PDF exclusivo para André Pires de Oliveira Capela - rm376905
andr.capela@gmail.com

No entanto, apesar de serem mais eficientes que as MLPs, as CNNs ainda exigem uma quantidade considerável de recursos computacionais, especialmente quando aplicadas a conjuntos de dados de grande escala ou em tarefas de alta complexidade. Uma ideia geral do tamanho de algumas arquiteturas de CNNs treinadas para análise de imagens pode ser vista na Figura 1 – “Número de parâmetros de algumas arquiteturas de CNNs”:

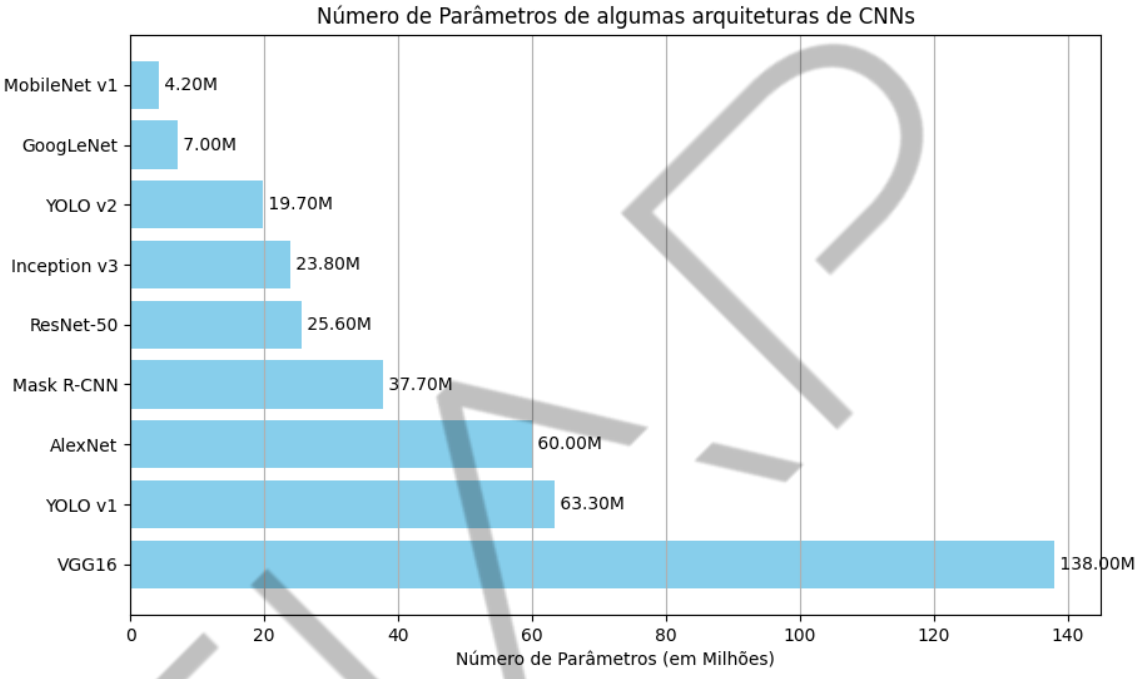


Figura 1 - Número de parâmetros de algumas arquiteturas de CNNs
Fonte: elaborado pelo autor (2023), adaptado por FIAP (2024)

Essa quantidade de parâmetros se traduz computacionalmente em uso de memória, geralmente de GPU, para treinamento e quantidade de operações de ponto flutuante necessárias para realizar o treinamento, expressa em GFLOP (Giga Floating-point Operations ou bilhões de operações de ponto flutuante).

Embora a quantidade de memória necessária para o treinamento dessas CNNs dependa de uma série de escolhas usadas para o treinamento, como, por exemplo, o tamanho das imagens utilizadas, podemos fazer uma aproximação a partir dos artigos publicados por seus desenvolvedores e um gráfico estimado, como o apresentado na Figura 2 – “Estimativa do uso de VRAM para treinamento de algumas arquiteturas de CNNs”:

PDF exclusivo para André Pires de Oliveira Capela - rm376905
andr.capela@gmail.com

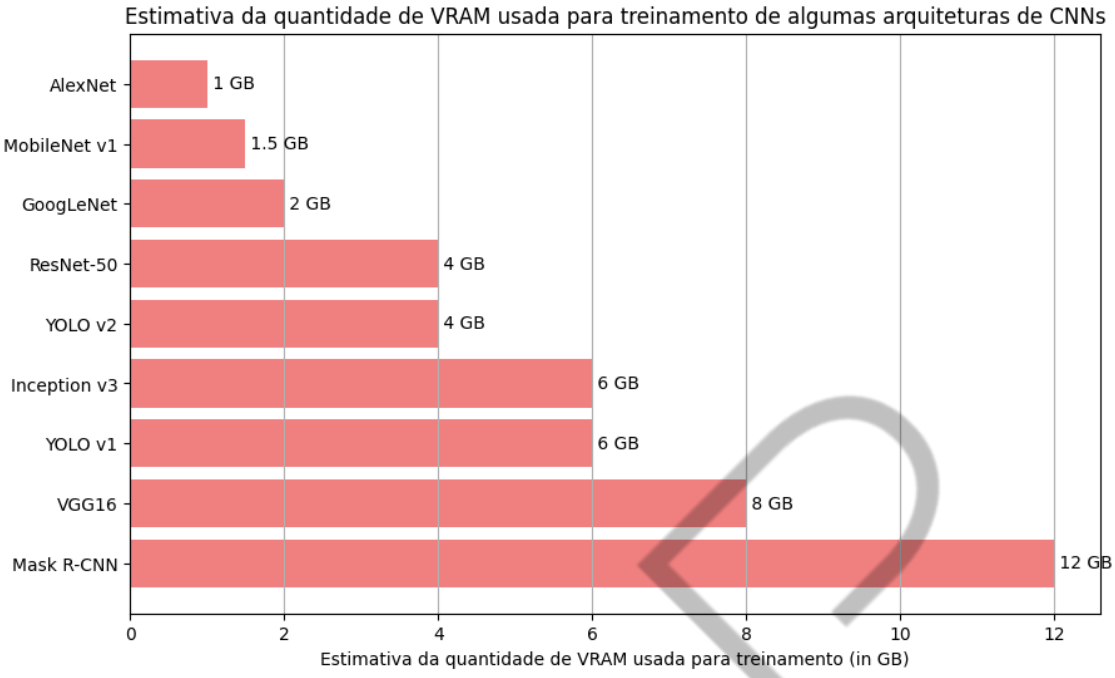


Figura 2 - Estimativa do uso de VRAM para treinamento de algumas arquiteturas de CNNs
Fonte: elaborado pelo autor (2023), adaptado por FIAP (2024)

Como podemos ver, não existe uma relação de 1 para 1 em termos de parâmetros e quantidade de memória necessária para o treinamento, pois essa quantidade é influenciada por diversas escolhas no treinamento. Por exemplo, um dos fatores que influencia bastante a quantidade de memória usada é o otimizador escolhido. Otimizadores como o Adam, que mantêm estimativas do primeiro e segundo momentos dos gradientes, exigem mais memória para armazenar essas informações adicionais para cada parâmetro da rede. Em contraste, otimizadores mais simples, como o SGD (Stochastic Gradient Descent), usam menos memória porque atualizam os parâmetros diretamente com base no gradiente atual, sem a necessidade de armazenar estimativas adicionais. Como estamos tratando de milhões de parâmetros, essa escolha faz uma enorme diferença para as CNNs.

Uma informação importante que pode ser obtida da Figura 2 é que, mesmo arquiteturas de CNNs mais complexas, como a MaskRCNN, que tem a capacidade de realizar segmentação de instância, podem ser treinadas em sistemas com uma única GPU. Esse é um item importante que vai guiar nossa discussão: se uma rede neural precisa de mais memória de GPU do que você tem disponível, você não vai conseguir treiná-la. Até existem métodos de swap para VRAM, mas eles não funcionam bem o suficiente para o uso em produção. Então VRAM é um limitador rígido da nossa

PDF exclusivo para André Pires de Oliveira Capela - rm376905
andr.capela@gmail.com

escolha. A velocidade de processamento, como veremos a seguir, é um limitador mais elástico, que depende apenas de atender os requisitos de tempo da sua aplicação. Dessa forma, é possível fazer tanto treinamento quanto inferência em qualquer uma das CNNs apresentadas em sistemas com apenas uma GPU, desde que ela tenha ao menos 12GB para o pior cenário, permitindo o uso até mesmo de placas de vídeo voltadas para jogos, como as RTX 4060 Ti e RTX 4070 para isso.

Do ponto de vista de processamento necessário, podemos nos basear também nos trabalhos publicados, sempre dividindo o tempo pelo número de epochs usadas e quantidade de imagens no dataset, para ter uma comparação justa. O resultado pode ser visto na Figura 3.

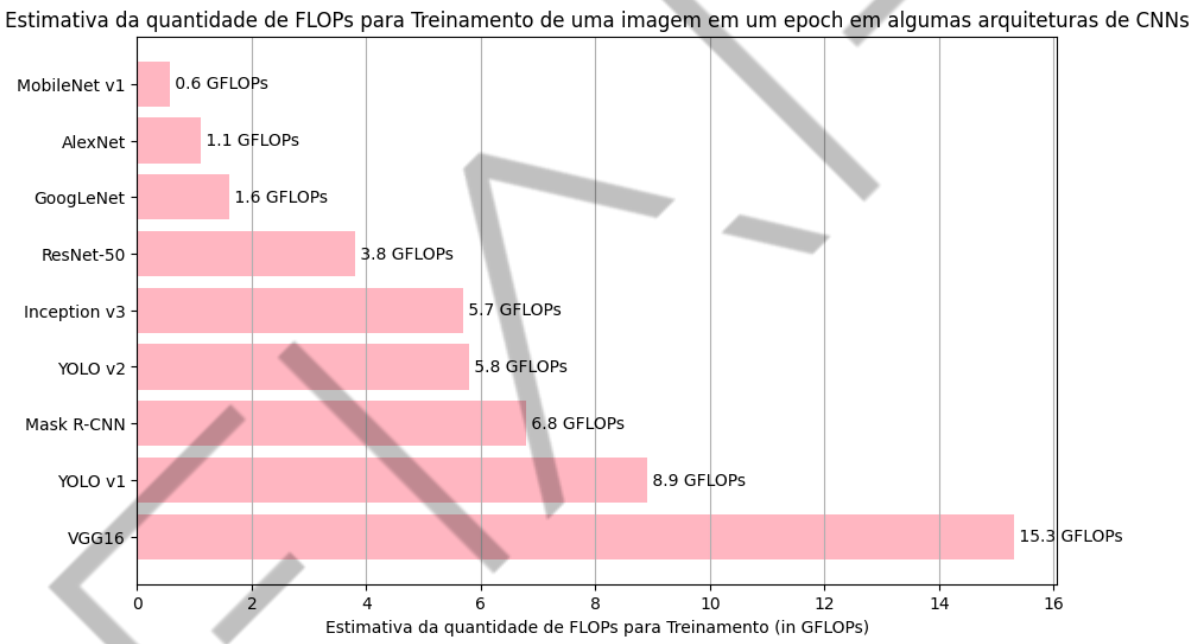


Figura 3 - Estimativa do uso de FLOPs para treinamento de algumas arquiteturas de CNNs
Fonte: elaborado pelo autor (2024)

Para entender esse gráfico, é necessário retornar um pouco e discutir qual é a capacidade de processamento de ponto flutuante de processadores e placas de vídeo.

Desde o lançamento do Intel 486, em abril de 1989, todos os processadores com arquitetura x86 possuem capacidade de realizar diretamente operações matemáticas com variáveis de ponto flutuante (conhecidas como float em linguagens de programação). No início, os processadores tinham capacidade de realizar uma operação de ponto flutuante por clock, dessa forma, um 486 de 100MHz tinha capacidade de processar 0.1 GFLOP/s [1]. Isso começou a mudar com a adição de mais instruções no processador, que permitiam combinar operações dentro de um

PDF exclusivo para André Pires de Oliveira Capela - rm376905
andr.capela@gmail.com

mesmo ciclo de clock. Comercialmente, a primeira adição de instruções ficou conhecida como MMX, introduzida em 1997 na arquitetura Pentium (P5). Mas as instruções MMX só lidavam com inteiros, não modificando a capacidade de processamento de ponto flutuante. A verdadeira mudança veio com as instruções 3DNow em processadores da AMD em 1998 e SSE para processadores da Intel em 1999. Por esse motivo, um Pentium III de 1GHz, que já contém as instruções SSE, tem capacidade de 1,4 GFLOP/s [1]. Isso melhorou ainda mais com as adições subsequentes das instruções SSE2, SSE3, SSE4, AVX, AVX2, AVX512 e, no futuro, AVX10.

Por sorte não estamos mais na era do Pentium III, caso contrário, levaríamos quase 11 segundos por foto no treinamento de uma rede VGG. Se considerarmos que o dataset do ImageNet Large Scale Visual Recognition Challenge (ILSVRC) tem 1,2 milhões de imagens, um Pentium III de 1GHz levaria quase 153 dias por epoch de treinamento. Considerando 100 epochs de treinamento como padrão, teríamos 42 anos para o treinamento completo. Dessa forma, se alguém começou esse treinamento no lançamento do Pentium III, está aproximadamente em 50% dele hoje.

Nesse momento, já está claro que houve uma enorme evolução na capacidade de cálculo dos processadores nas últimas décadas: do 486 ao Pentium III, houve um aumento de 14 vezes em 6 anos (entre 1994 e 2000). Nesse ponto, vamos introduzir um modelo preditivo para essa evolução: a Lei de Moore.

A Lei de Moore, formulada por Gordon Moore, cofundador da Intel, em 1965, é uma observação empírica sobre o desenvolvimento da tecnologia de semicondutores. Moore notou que o número de transistores em microchips dobrava aproximadamente a cada ano, sugerindo um ritmo exponencial de crescimento para a capacidade de processamento e eficiência dos dispositivos eletrônicos. A previsão inicial de Moore foi feita com base na análise das tendências da indústria na época e ele previu que essa tendência continuaria por, pelo menos, mais uma década.

Com o passar dos anos, a Lei de Moore foi revisada pelo próprio Moore e, em 1975, ajustou a duplicação para aproximadamente a cada dois anos, refletindo mais precisamente o ritmo do avanço tecnológico observado na produção de circuitos integrados. Essa lei tornou-se um princípio orientador para a indústria de semicondutores, impulsionando a inovação e o investimento em pesquisa e

PDF exclusivo para André Pires de Oliveira Capela - rm376905
andr.capela@gmail.com

desenvolvimento para manter o ritmo de miniaturização e aumento de desempenho dos chips.

Embora originalmente focada no número de transistores em microchips, essa lei tem uma relação direta com a capacidade de processamento dos processadores. À medida que o número de transistores em um chip dobra, conforme observado por Moore, espera-se que a capacidade de processamento também aumente. Isso ocorre porque mais transistores significa que mais operações lógicas podem ser realizadas simultaneamente, permitindo que os processadores executem mais instruções por ciclo de clock ou tenham mais núcleos de processamento, o que resulta em um desempenho geral melhor.

A Lei de Moore não é uma lei baseada na física ou matemática, é uma previsão baseada em tendências econômicas e tecnológicas que tem se mantido relevante por várias décadas. No entanto, nos anos recentes, a indústria começou a enfrentar limitações físicas e desafios tecnológicos que tornam cada vez mais difícil manter esse ritmo de escalonamento para capacidade de processamento. O primeiro desafio foi pelas frequências de operação, que continuaram a ser aumentadas até chegar aproximadamente aos 4 GHz. Nesse ponto, o vazamento de corrente, que se traduz em uma maior geração de calor nos processadores, se tornou um limitante, levando ao desenvolvimento de processadores com mais núcleos no lugar de núcleos com maior frequência. Os limitadores atuais são de outra natureza, principalmente devido aos processos litográficos cada vez menores, como a litografia ultravioleta extrema (EUV). Isso inclui desafios como efeitos quânticos indesejados que ocorrem em escalas extremamente pequenas, como os atuais quatro (4) nanômetros, e o aumento dos custos de fabricação.

Mesmo com os desafios atuais, vemos que a Lei de Moore tem se mantido razoável nas previsões de capacidade de processamento. Uma visão dessa evolução na última década, dentro da linha de processadores “Core” da Intel, pode ser vista em uma estimativa compilada na Tabela 1 – Estimativa da capacidade de processamento de processadores modernos.

Modelo	Ano de lançamento	Capacidade em GFLOP/s	Preço em dólar (corrigido para 2023)
Core i7-6700K	2015	256	419
Core i7-8700K	2017	355.2	414

PDF exclusivo para André Pires de Oliveira Capela - rm376905
andr.capela@gmail.com

Core i7-10700K	2020	486.4	420
Core i7-12700K	2021	691.2	434
Core i7-14700K	2023	957.4	409

Tabela 1 - Estimativa da capacidade de processamento de processadores modernos

Embora tenha um aumento notável na capacidade de processamento, a variável não segue exatamente a Lei de Moore, pois dobrar a quantidade de transistores não se traduz diretamente em dobrar a capacidade de processamento, especialmente considerando processadores atuais, que contam com diversos outros sistemas dentro deles, como o controlador de memória e, muitas vezes, até mesmo uma GPU. Mesmo assim, nesses 8 anos, da geração 6 até a 14 da Intel, houve um aumento de 3,7 vezes na capacidade de processamento.

Outro fator interessante da tabela, que até épocas recentes se estendia também para GPUs, é que o valor corrigido pela inflação (dos Estados Unidos) de cada nova geração de processador se mantém relativamente constante.

Voltando ao nosso tópico inicial, quanto tempo é necessário para treinar uma rede neural VGG16 usando um processador como o i7-14700k? Dessa vez, o tempo necessário é de aproximadamente 22 dias, o que é consideravelmente melhor do que 42 anos, mas ainda bastante longe do que é possível de se obter ao usar servidores para essa tarefa. Até aqui, usamos apenas números de desempenho de processadores para usuários domésticos, sem incluir servidores ou GPUs.

Um servidor contendo um Intel Xeon 8593Q ou um AMD EPYC 9754 terá um desempenho entre 3 e 4 vezes melhor, baixando esse tempo para menos de uma semana. Isso considerando apenas uma CPU, o que é bastante incomum ao se tratar de servidores: geralmente, cada placa-mãe suporta múltiplos processadores e o parque computacional geralmente consiste em apenas um cluster de servidores.

Embora servidores tenham capacidade para treinar CNNs, nesse cenário as GPUs demonstram uma superioridade computacional que só é rivalizada por sistemas especificamente criados para lidar com Redes Neurais, como processadores tensoriais (TPUs). Uma GPU de servidor como a H100 SXM pode realizar essa mesma tarefa em pouco mais de 30 minutos, se utilizadas otimizações nas variáveis, ou em menos de 8 horas, sem otimização alguma no código. Mesmo uma GPU de

PDF exclusivo para André Pires de Oliveira Capela - rm376905
andr.capela@gmail.com

uso gamer, como a 4060 Ti, consegue fazê-lo em pouco mais de 23 horas, supondo o uso de precisão simples (float 32 bits) e nenhuma outra otimização.

Com esses números em mente, podemos voltar ao nosso tema principal: como se comportam os Large Language Models (LLMs) mais conhecidos em relação ao tamanho de memória necessário para o treinamento? Para isso, vamos examinar a quantidade de parâmetros utilizada por alguns LLMs conhecidos.

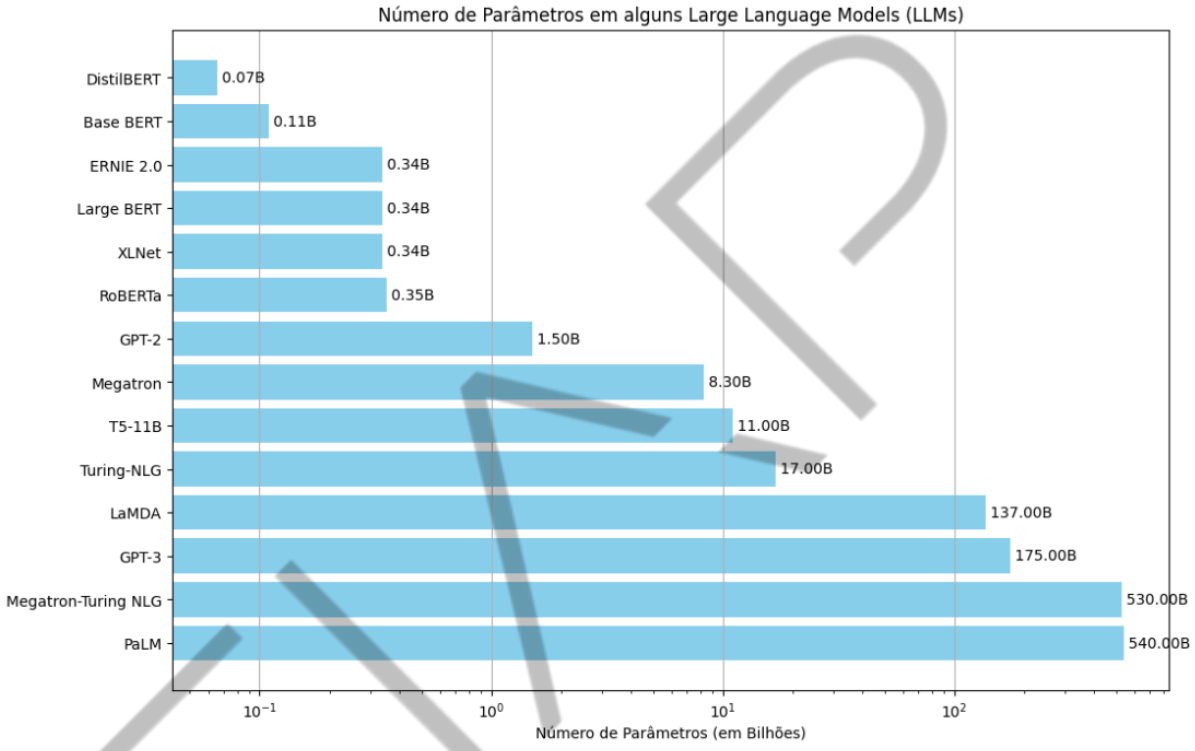


Figura 4 - Número de parâmetros de algumas arquiteturas de LLMs
Fonte: elaborado pelo autor (2023), adaptado por FIAP (2024)

Uma observação importante é que, diferente dos outros gráficos exibidos anteriormente, a Figura 4 – “Número de parâmetros de algumas arquiteturas de LLMs” se encontra em escala logarítmica, pois contém modelos com diferenças de 4 ordens de magnitude na quantidade de parâmetros. Pode-se observar nesse gráfico que os dois primeiros modelos, “DistilBERT” e “Base BERT”, possuem quantidades de parâmetros comparáveis com os modelos de CNNs que estávamos discutindo. Logo, tudo que discutimos anteriormente se aplica a eles também.

Estimar a quantidade necessária de memória RAM de GPU para o treinamento de LLMs é bem mais complicado do que com CNNs e depende ainda mais de escolhas de otimização durante o treinamento. Além disso, nem todos os modelos são abertos. Da arquitetura do GPT-3, por exemplo, só se conhece a quantidade de parâmetros; do GPT-4, nem mesmo isso é conhecido, por isso ele não está na lista. Uma estimativa

PDF exclusivo para André Pires de Oliveira Capela - rm376905
andr.capela@gmail.com

do mínimo de memória necessária pode ser vista na Figura 5 – “Estimativa do uso de VRAM para treinamento de algumas arquiteturas de LLMs”.

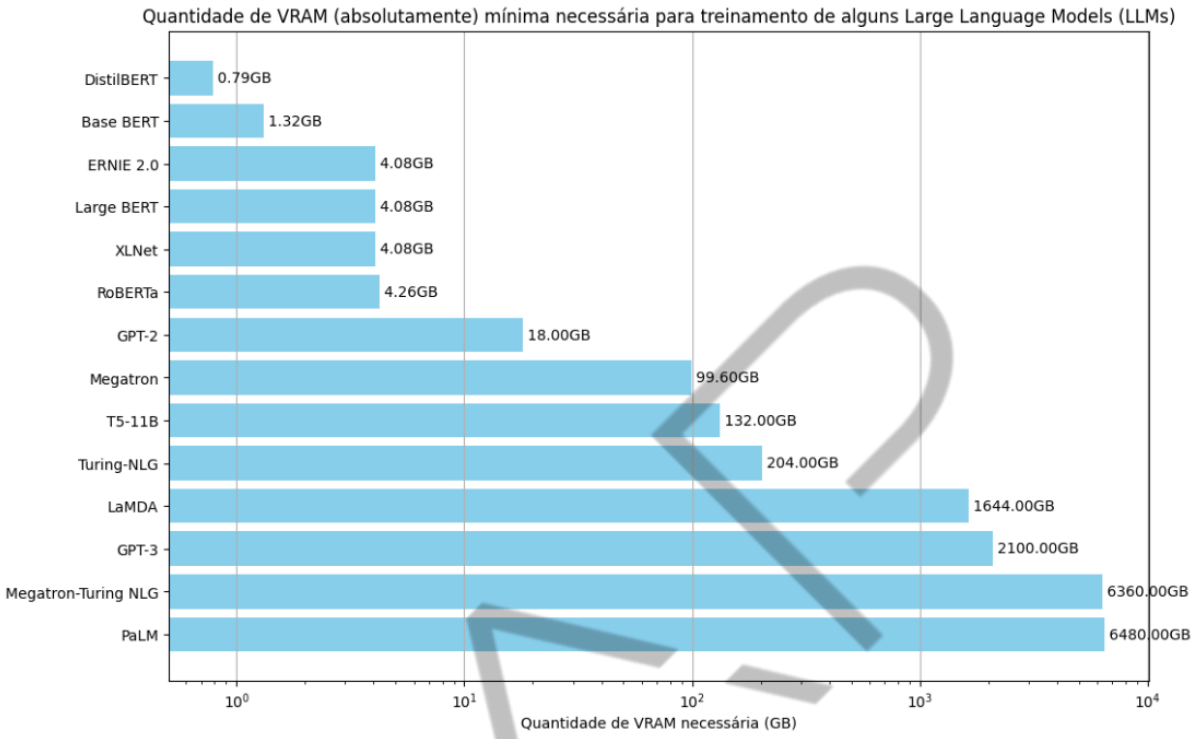


Figura 5 - Estimativa do uso de VRAM para treinamento de algumas arquiteturas de LLMs
Fonte: elaborado pelo autor (2024)

Esse gráfico nos permite classificar as LLMs em 3 regiões distintas. A primeira região abrange do modelo DistilBERT até o GPT-2, contendo os modelos que podem ser treinados em uma única GPU. Para essa comparação, vamos supor que uma GPU gamer possui no máximo 24 GB de memória (referência: RTX 4090) e uma GPU para servidor possui no máximo 80 GB de memória (referência H100 PCIe). Dessa forma, treinar um modelo como o GPT-2 estaria bem próximo do limite de uma GPU de topo de linha gamer dessa geração.

A próxima região abrange desde o modelo Megatron até o Turing-NLG, contendo os modelos que podem ser treinados uma única máquina com múltiplas GPUs. Para isso, vamos considerar um máximo usual de 4 GPUs em um único servidor, tendo um total de 320 GB de VRAM. Essa região já exclui a utilização de GPUs gamer para treinamento, sendo necessário o uso de GPUs produzidas para essa finalidade em servidores, com linhas PCIe suficientes para essa demanda.

Por último temos os modelos LaMDA e maiores, que só podem ser treinados usando um cluster de servidores com GPUs. Nessa região nós temos, por exemplo, o

PDF exclusivo para André Pires de Oliveira Capela - rm376905
andr.capela@gmail.com

modelo PaLM, desenvolvido pelo Google, que necessita de, ao menos, 6TB de VRAM para o treinamento. Diversos outros modelos, que não estão no gráfico, caem nessa mesma classe, como o PaLM2, o LLama70b e muitos outros que temos hoje. Usando como base os resultados de desempenho o modelo GPT-4 da OpenAI, ele certamente se encontra nessa classe também, embora seja impossível estimar o uso de recursos, por ser um modelo de código fechado.

Usar alguns Terabytes de VRAM não é algo ao alcance da maioria dos usuários, mesmo de usuários com acesso a datacenters de empresas e universidades. Isso levou a um aumento enorme na demanda por GPUs nos últimos 2 anos: somente em 2023, estima-se que a NVIDIA tenha enviado 650 mil GPUs H100. Em 2024, estima-se uma venda de 1,5 a 2 milhões de unidades dessa mesma GPU. Para colocar esse número em perspectiva, se todas as unidades forem ligadas ao mesmo tempo com o seu consumo máximo (700W por unidade), seria necessário um pouco mais do que a potência de uma usina nuclear, como a de Angra II, para gerar a energia necessária.

Com todo esse custo, fica a pergunta: modelos tão grandes são realmente necessários? No caso da tecnologia de Transformers usada em LLMs, a resposta é sim! Uma animação feita pelo Google para o modelo PaLM ilustra muito bem esse conceito.

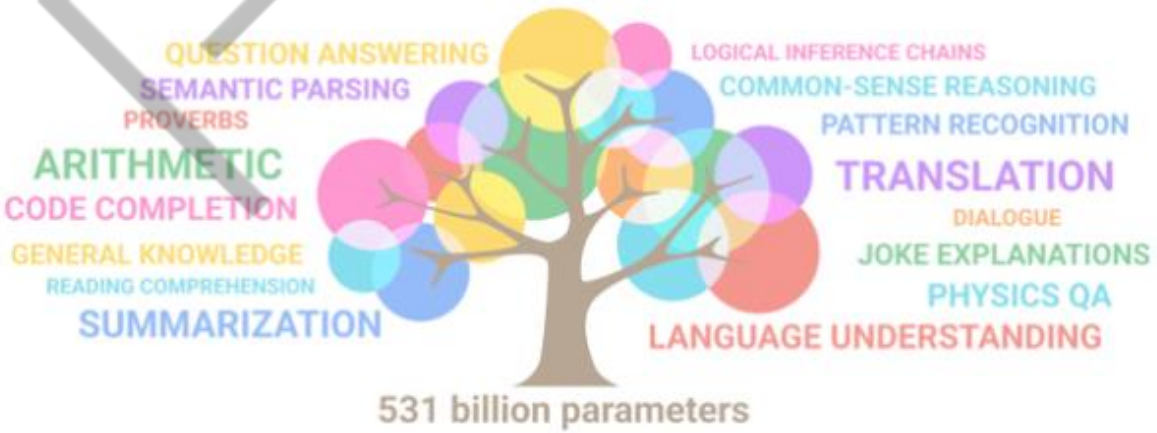


Figura 6 - Animação feita pelo Google ilustrando a capacidade pela quantidade de parâmetros típica de um LLM
Fonte: Google [s.d]

PDF exclusivo para André Pires de Oliveira Capela - rm376905
andr.capela@gmail.com

Até o momento, estamos preocupados apenas com o tamanho dos modelos, que é reflexo direto da quantidade de parâmetros. Como discutimos anteriormente, esse é um limitante rígido para o treinamento: você precisa ter a quantidade de memória mínima, caso contrário, não treinará o modelo. Agora, vamos verificar o tempo necessário para o treinamento em um datacenter.

Ao tratarmos de LLMs, a estratégia usada no treinamento tem um peso muito grande na quantidade de operações de ponto flutuante necessárias para o treinamento. Em particular, a quantidade de tokens de texto tem um papel crucial, sendo essa escolha discutida em diversos artigos publicados no último ano. Tomando como base o artigo “Training Compute-Optimal Large Language Models”, de 2022, da empresa DeepMind, temos a relação ótima entre FLOPs e quantidade de parâmetros para uma LLM, que está expressa na Figura 7 – “Previsões de modelos para treinamentos de LLMs em FLOPs por parâmetros”:

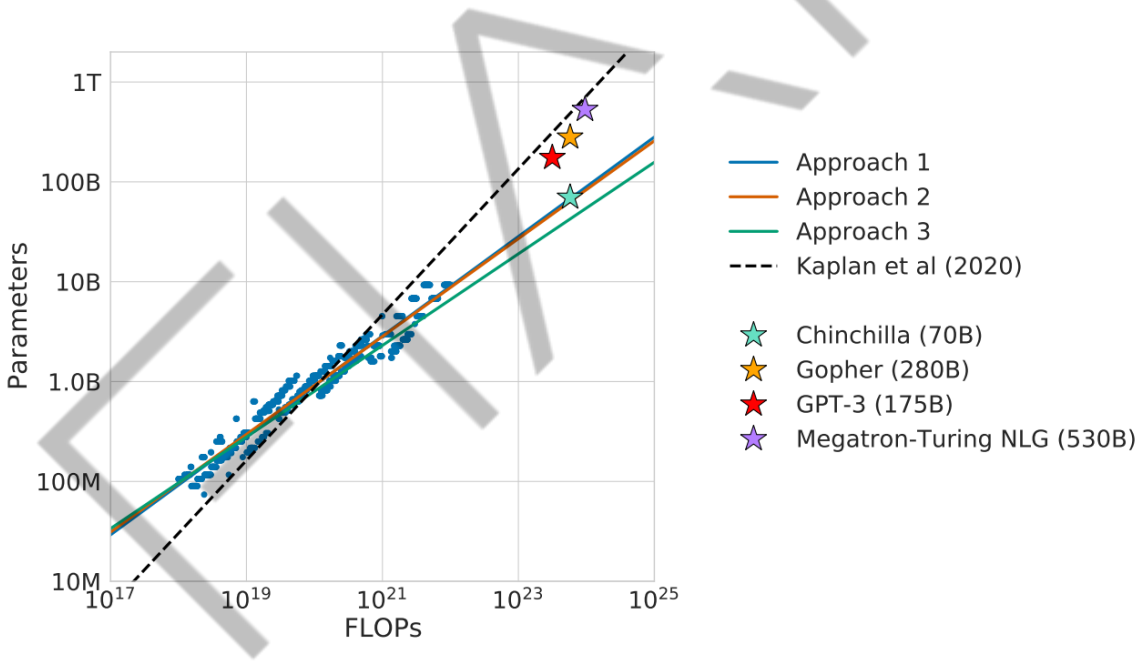


Figura 7 - Previsões de modelos para treinamento de LLMs em FLOPs por parâmetros
Fonte: elaborado pelo autor (2023), adaptado por FIAP (2024)

Pode-se observar que, tipicamente, um modelo com 10 bilhões de parâmetros vai precisar de algo entre 10^{21} e 10^{22} FLOPs para ser treinado. Vamos trabalhar com esse valor para o cálculo de tempo de treinamento. Considere que, para isso, vamos usar um datacenter de pequeno ou médio porte, com 256 GPUs de geração anterior, a NVIDIA A100, com 40GB cada (para diminuir nosso custo). Cada uma dessas GPUs tem capacidade de 19.5 TFLOP/s, fazendo com que o treinamento, supondo nenhum

PDF exclusivo para André Pires de Oliveira Capela - rm376905
andr.capela@gmail.com

outro gargalo, utilize entre 2 e 7 dias dependendo da previsão utilizada. Como podemos ver nesse exemplo, mesmo utilizando 256 GPUs e um modelo muito menor que o estado da arte, um treinamento como esse pode levar uma semana e, usando esses recursos, custar em torno de 100.000,00 dólares em um provedor de nuvem [6].

Hoje, o treinamento do estado da arte em modelos de LLMs é algo que só é possível em grandes empresas e centros de pesquisa. Mas será que esse cenário mudará rapidamente? Vamos reexaminar a Lei de Moore para procurar uma resposta.

Embora atualmente a Lei de Moore não seja traduzida diretamente em dobrar o desempenho a cada 2 anos, vamos adotar esse comportamento como um possível guia. Também vamos adotar um comportamento que observamos na Tabela 1 – “Estimativa da capacidade de processamento de processadores modernos”, considerando que novas tecnologias tendem a custar o mesmo das anteriores, desde que corrigido pela inflação.

Seguindo esse modelo, deve ser possível treinar uma LLM de 10 bilhões de parâmetros em um servidor com 4 GPUs em uma semana no ano de 2035. Por outro lado, pode-se treinar na nuvem a um custo de 12,5 mil dólares já em 2029. Vale lembrar que estamos falando de 10 bilhões de parâmetros, ou seja, uma rede 17,5 vezes menor que o GPT-3, que não é mais o estado da arte mesmo em 2024. Por esse motivo, treinar o estado da arte dos modelos de LLMs deve continuar exclusivo aos datacenters por ao menos algumas décadas, considerando as técnicas de treinamento atuais.

Isso significa que é impossível treinar e fazer inferência com LLMs em um PC doméstico? Para responder isso, vamos rever a figura 5 – “Estimativa do uso de VRAM para treinamento de algumas arquiteturas de LLMs”. Nessa figura, podemos ver que a primeira classe de modelos usa até 18 GB de VRAM. Desses modelos, você pode escolher para testar, tanto em casa quanto no Google Colab, o modelo “RoBERTa”, que pode ser obtido facilmente por meio do modelo “Clip” (Contrastive Language-Image Pre-Training). Usando as informações contidas no Github, vocês conseguem rapidamente testar inferência usando o modelo e até mesmo iniciar um treinamento dele, mesmo com recursos computacionais limitados. É importante também observar que esse trabalho está sendo usado como base para muitos outros, como o modelo CAPIVARA, que tem como objetivo treinar uma LLM multimodal especializada em português do Brasil.

FIAP

PDF exclusivo para André Pires de Oliveira Capela - rm376905
andr.capela@gmail.com

Se você conseguiu testar algum dos modelos disponíveis no Clip ou ficou com alguma dúvida no conteúdo ou implementação, venha conversar e trocar informações no Discord. Vemos vocês lá!

FIAP

PDF exclusivo para André Pires de Oliveira Capela - rm376905
andr.capela@gmail.com

O QUE VOCÊ VIU NESTA AULA?

Nesta aula, exploramos a implementação e o impacto dos Grandes Modelos de Linguagem (LLMs) em aplicações práticas, um avanço crucial na fronteira da inteligência artificial. Enfatizamos o caminho percorrido desde os conceitos básicos até a adoção de LLMs em projetos reais, ilustrando a evolução dessa tecnologia desde suas raízes históricas até sua aplicação contemporânea em assistentes virtuais e chatbots. Discutimos as três principais abordagens para a integração de LLMs: a execução local (On-Premises), para organizações que valorizam a segurança e a privacidade, a alocação em servidores de nuvem, para aqueles que buscam escalabilidade, e o uso de APIs de modelos pré-treinados, uma solução prática para superar desafios de custo e complexidade computacional.

Destacamos, também, o desafio significativo representado pelo alto custo de treinamento e inferência com LLMs, apontando que a maioria das empresas, especialmente no Brasil, enfrenta limitações para adotar essas tecnologias de forma eficaz. Como resposta a essas limitações, a aula se concentrou no uso de APIs como um meio de acessar capacidades avançadas de processamento de linguagem natural, permitindo aos desenvolvedores e desenvolvedoras criarem soluções inovadoras sem a necessidade de uma infraestrutura computacional avançada.

FIAP

PDF exclusivo para André Pires de Oliveira Capela - rm376905
andr.capela@gmail.com

REFERÊNCIAS

HOFFMANN, J et. al. **Training Compute-Optimal Large Language Models**. 2022. Disponível em: https://arxiv.org/abs/2203.15556. Acesso em: 01 abr. 2024.

SANTOS, G. et al. **CAPIVARA**: **Cost-Efficient Approach for Improving Multilingual CLIP Performance on Low-Resource Languages**, 2023. Disponível em: https://arxiv.org/abs/2310.13683v2. Acesso em: 01 abr. 2024.

VASWANI, A. et al. **Attention is all you need**. 2017. Disponível em: <https://arxiv.org/abs/1706.03762>. Acesso em: 01 abr. 2024.

PDF exclusivo para André Pires de Oliveira Capela - rm376905
andr.capela@gmail.com

PALAVRAS-CHAVE

LLMs. CNNs. FLOPs. GPU. CPU. Custo de Operação.

FIAP

PDF exclusivo para André Pires de Oliveira Capela - rm376905
andr.capela@gmail.com



PDF exclusivo para André Pires de Oliveira Capela - rm376905
andr.capela@gmail.com